

Injectics

Relatório de Invasão: Injectics

Resumo

Objetivo: Explorar a aplicação web injectics

Vetor de Acesso Inicial: SQL injection no form de login .

Vetor de Elevação de Privilégios: Dropar table users e conseguir logar com super admin com suas credenciais padrões e depois abusar de um camp que permitia SSTI e assim logarmos no servidor como www-data.

Ferramentas & Scripts Utilizados

- nmap
- ffuf

1. Reconhecimento & Mapeamento

Scan de Portas (Nmap)

Output do Nmap -sS

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo nmap -sS 10.66.178.187 -p1-10000
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-28 19:21 -0400
Nmap scan report for 10.66.178.187
Host is up (0.14s latency).
Not shown: 9998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 48.95 seconds
```

☰ Output do Nmap -sU

```
(kali@kali)-[~]
└─$ sudo nmap -sU 10.66.178.187 -F
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-28 19:23 -0400
Nmap scan report for 10.66.178.187
Host is up (0.16s latency).
Not shown: 98 closed udp ports (port-unreach)
PORT      STATE      SERVICE
68/udp    open|filtered dhcpc
4444/udp  open|filtered krb524

Nmap done: 1 IP address (1 host up) scanned in 108.45 seconds
```

☰ Output do Nmap com -sC e -sV

```
└─$ sudo nmap -sV -sC 10.66.178.187 -p1-10000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-28 19:36 -0400
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 1 undergoing Ping Scan
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 10.66.178.187
Host is up (0.14s latency).
Not shown: 9998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 e9:9f:8f:2b:38:fd:63:f2:c3:e3:d0:16:6d:9c:1b:5d (RSA)
|   256 44:a3:39:f3:49:c8:eb:a3:83:fe:dc:5b:a5:7a:8e:88 (ECDSA)
|_  256 6e:fb:c9:63:68:44:c9:82:9e:10:a0:ba:6c:da:ac:e4 (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-cookie-flags:
|   /:
|     PHPSESSID:
|_    httponly flag not set
|_ http-title: Injectics Leaderboard
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 62.32 seconds
```

📘 Análise do Reconhecimento

Vemos aqui a presença de um host com ssh aberto na porta 22 e um servidor web na porta 80 indicando um servidor provavelmente php com apache (LAMP) rodando em um ubuntu server , sem http only permtindo roubo de cookies via XSS.

2. Enumeração Web & VHOSTs

```
gobuster dir -u http://[IP] -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,txt,log
```

🔗 Dados e reconhecimento

```
(kali@kali)-[/media/sf_pastaCompartilhadaVmKali/txtsuteis]
$ ffuf -u http://10.66.178.187/FUZZ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 100 -e .php,.php.bak,.log,.txt -fs 6588

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.66.178.187/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Extensions : .php .php.bak .log .txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 100
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response size: 6588

.php          [Status: 403, Size: 278, Words: 20, Lines: 10, Duration: 138ms]
login.php    [Status: 200, Size: 5401, Words: 1972, Lines: 162, Duration: 138ms]
mail.log     [Status: 200, Size: 1098, Words: 192, Lines: 24, Duration: 135ms]
flags       [Status: 301, Size: 314, Words: 20, Lines: 10, Duration: 134ms]
css         [Status: 301, Size: 312, Words: 20, Lines: 10, Duration: 136ms]
js         [Status: 301, Size: 311, Words: 20, Lines: 10, Duration: 135ms]
javascript  [Status: 301, Size: 319, Words: 20, Lines: 10, Duration: 135ms]
logout.php  [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 134ms]
vendor      [Status: 301, Size: 315, Words: 20, Lines: 10, Duration: 135ms]
dashboard.php [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 137ms]
functions.php [Status: 200, Size: 0, Words: 1, Lines: 1, Duration: 220ms]
phpmyadmin  [Status: 301, Size: 319, Words: 20, Lines: 10, Duration: 135ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

Página inicial

Welcome to the Injectics 2024
Catch all the action and track the performance of your favorite athletes and teams.

Leaderboard

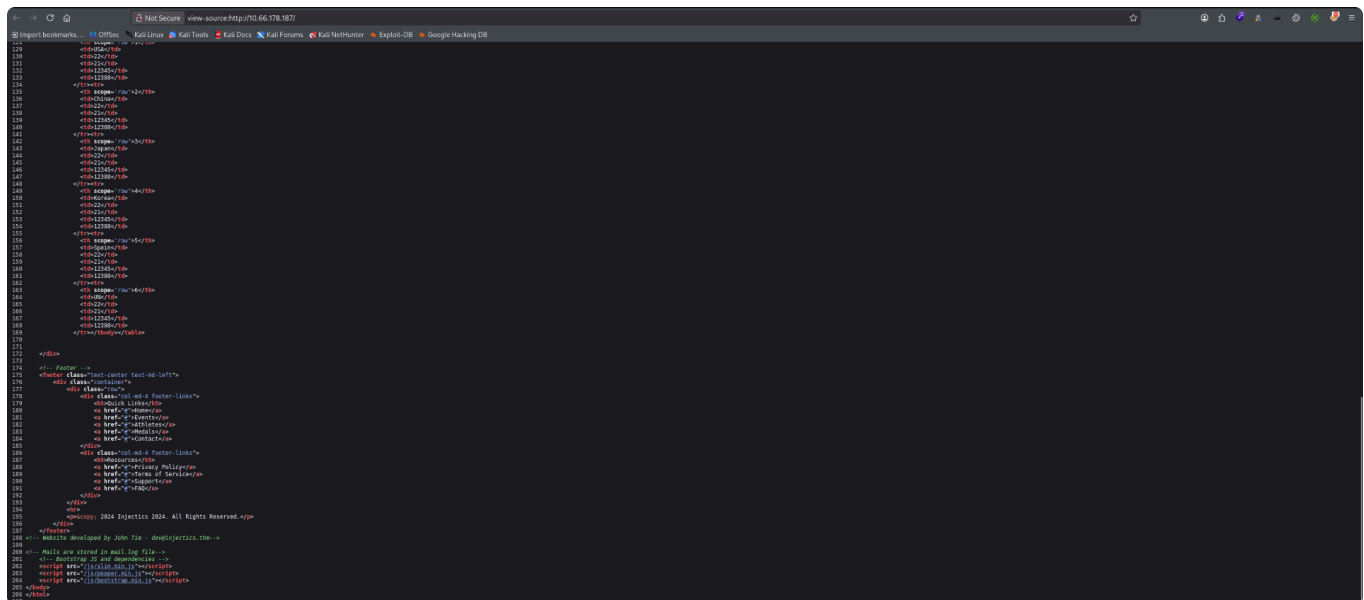
Rank	Country	Gold	Silver	Bronze	Total
1	USA	22	21	12345	12388
2	China	22	21	12345	12388
3	Japan	22	21	12345	12388
4	Korea	22	21	12345	12388
5	Spain	22	21	12345	12388
6	UN	22	21	12345	12388

Quick Links

- Home
- Events
- Athletes
- Medals
- Contact

Resources

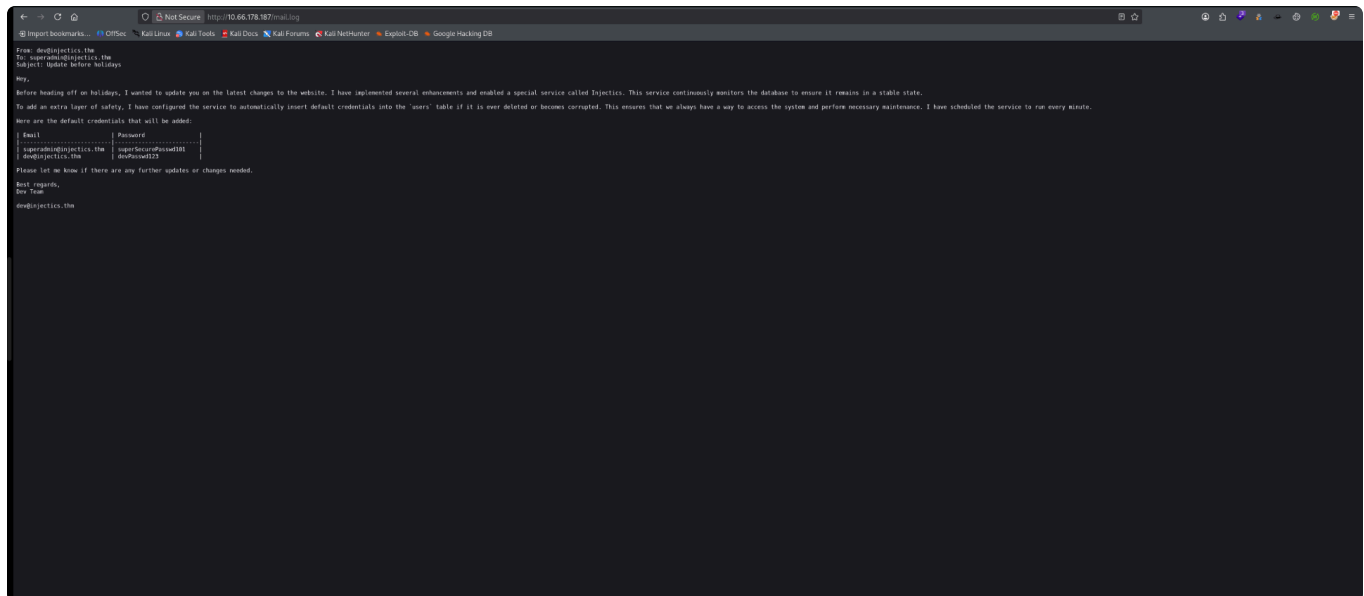
- Privacy Policy
- Terms of Service
- Support
- FAQ



Reconhecimento

Vemos que no page source de index.php temos informações expostas nele , como um email (dev@injectics.thm) e que os emails são guardados em mail.log.

mail.log



Reconhecimento

Aqui vemos a existência de dois emails sendo um deles já visto na page source da página inicial e agora vemos o email do superadmin. Porém essas credenciais foram mudadas , mas temos agora dois emails válidos para pode tentar fazer bypass de autenticação ou algo do tipo.Sabemos que tem um serviço rodando a cada minuto

monitorando para adicionar credenciais padrões nos users do mail.log caso os dados do banco sejam corrompidos ou deletados. Vemos também que o nome da tabela é users.

script.js

[illegible]

dashboard.php

Injectics

Logout

Welcome, dev!

Rank	Country	Gold	Silver	Bronze	Total	Actions
1	USA	1	21	12345	12367	Edit
2	China	1	21	12345	12367	Edit
3	Japan	1	21	12345	12367	Edit
4	Korea	1	21	12345	12367	Edit
5	Spain	1	21	12345	12367	Edit
6	UN	1	21	12345	12367	Edit

Quick Links

[Home](#)[Events](#)[Athletes](#)[Medals](#)[Contact](#)

Resources

[Privacy Policy](#)[Terms of Service](#)[Support](#)[FAQ](#)

© 2024 Injectics 2024. All Rights Reserved.

edit_leaderboard.php

The screenshot shows the 'Edit Leaderboard Entry' form in the Injectics application. The form is titled 'Edit Leaderboard Entry' and contains three input fields for medal counts: 'Gold' (value: 1), 'Silver' (value: 21), and 'Bronze' (value: 12345). Below these fields is a blue 'Update' button. The form is set against a light blue background. At the bottom of the page, there is a dark blue footer containing 'Quick Links' (Home, Events, Athletes, Medals, Contact) and 'Resources' (Privacy Policy, Terms of Service, Support, FAQ). The copyright notice '© 2024 Injectics 2024. All Rights Reserved.' is also present.

Reconhecimento

Aqui vemos uma rota de update para os registros dessa tabela no banco que faz controle da quantidade de medalhas de ouro , prata e bronze para um certo país. Essa rota é vulnerável a sql injection permitindo que dropemos a tabela users e assim loguemos com a credencial padrão do superadmin.

update_profile.php

The screenshot shows the 'Update Profile' form in the Injectics application. The form is titled 'Update Profile' and contains three input fields for user information: 'Email' (value: superadmin@injectics.thm), 'First Name' (value: admin), and 'Last Name' (empty). Below these fields is a blue 'Submit' button. The form is set against a light blue background. At the bottom of the page, there is a dark blue footer containing 'Quick Links' (Home, Events, Athletes, Medals, Contact) and 'Resources' (Privacy Policy, Terms of Service, Support, FAQ). The copyright notice '© 2024 Injectics 2024. All Rights Reserved.' is also present.

Reconhecimento

Essa rota se mostrou vulnerável a SSTI ao permitirmos violar o twig e executarmos um payload de reverse shell.

⚡ 3. Exploração & Acesso Inicial (Foothold)

✖ Vulnerabilidade encontrada

Título: Injeção SQL

Risk Rating: 10 DREAD

Resumo: O form de login possui vulnerabilidade de SQL injection onde somente certos filtros são aplicados na parte do cliente em forma de black list, assim não filtrando tudo.

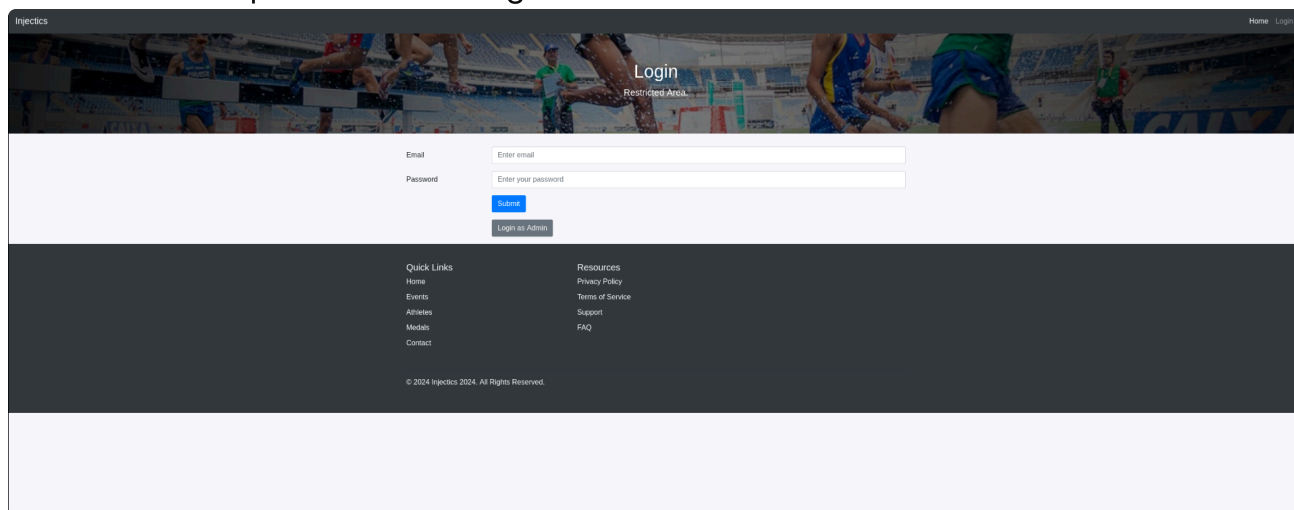
Background(Contexto adicional): SQL injection é a capacidade de fazer uma entrada ,em um campo que recebe entrada de usuário, ser parte de uma consulta SQL podendo bypassar autenticação , recuperar dados e demais coisas.

Impacto: Podemos logar sem utilizar das credenciais de login , permitindo logar e recuperar dados do usuário retornado e/ou executar solicitações em seu nome ao estar em sua conta.

Conselhos e Remediações: Não utilizar de consultar sql em formato raw e sim usar solicitações statement com bind_param.

Detalhes Técnicos e Evidências:

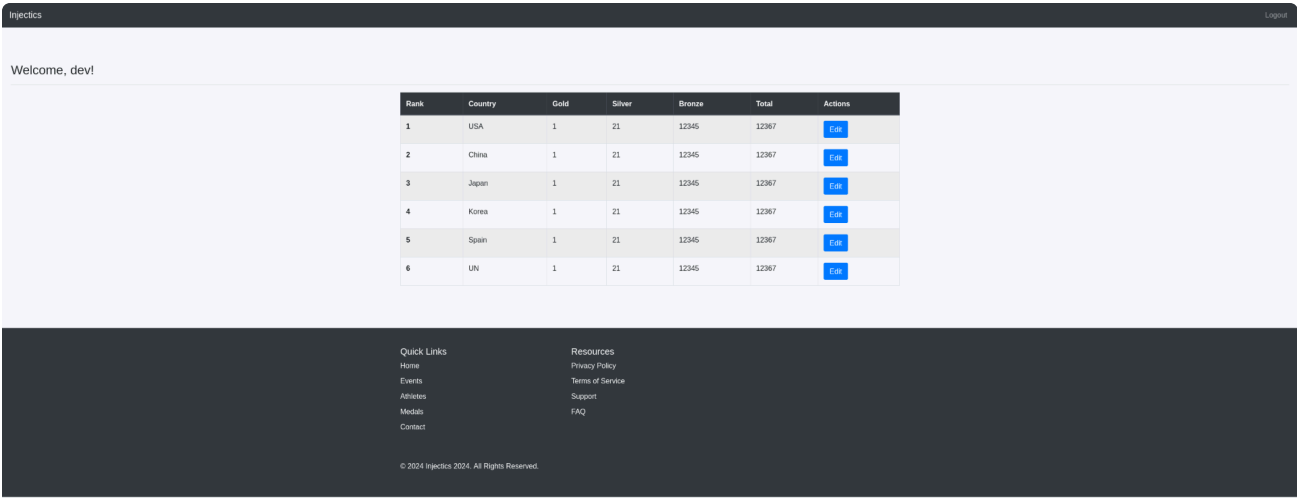
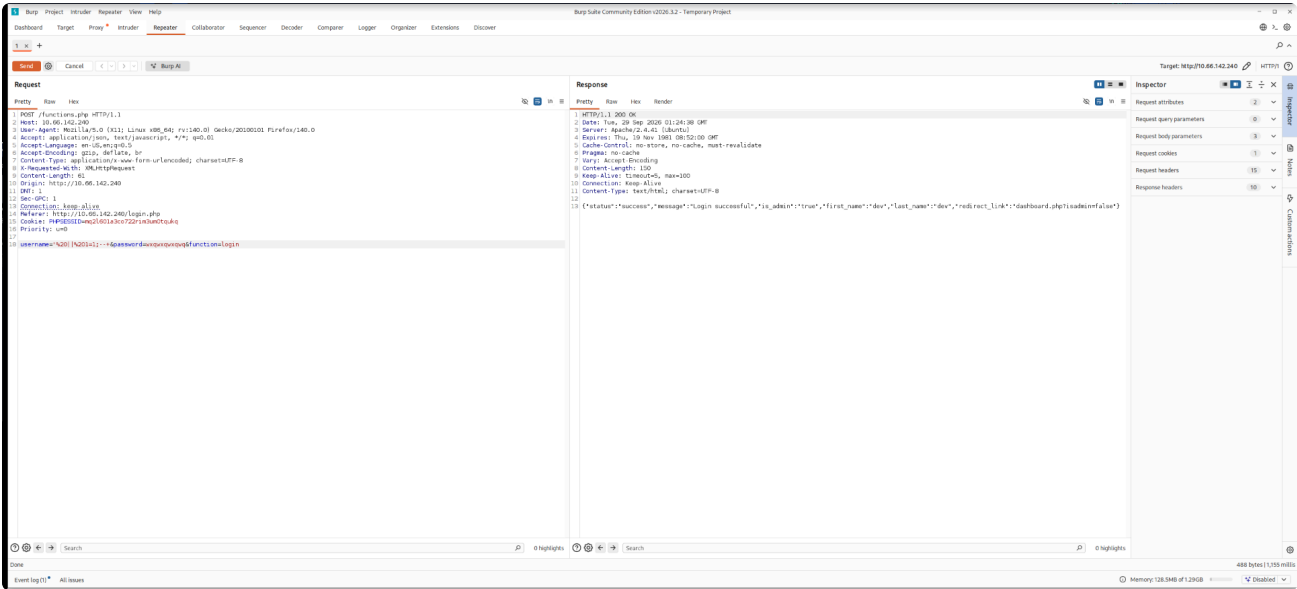
1. Primeiro vamos para o form de login.



The screenshot displays the login interface of a web application named 'Injectics'. The page features a header with the site name and navigation links. The main content area contains a login form with two input fields: 'Email' and 'Password'. Below these fields is a blue 'Submit' button and a link for 'Login as Admin'. The footer section includes 'Quick Links' (Home, Events, Admins, Media, Contact) and 'Resources' (Privacy Policy, Terms of Service, Support, FAQ). A copyright notice at the bottom reads '© 2024 Injectics 2024. All Rights Reserved'.

2. Depois colocamos qualquer coisa no form de login com o burp ouvindo no proxy e mandamos a request , salvamos ela no repeater e mandamos esse payload no email.

Assim fazemos login como user dev.



✗ Vulnerabilidade encontrada

Título: SQL injection

Risk Rating: 10 DREAD

Resumo: Foi encontrado um sql injection na rota de edit_leaderboard.php onde podemos executar um sql injection nos campos de form para update do registro no banco de dados.

Background(Contexto adicional): SQL injection é a capacidade de fazer uma entrada ,em um campo que recebe entrada de usuário, ser parte de uma consulta SQL podendo bypassar autenticação , recuperar dados e demais coisas.

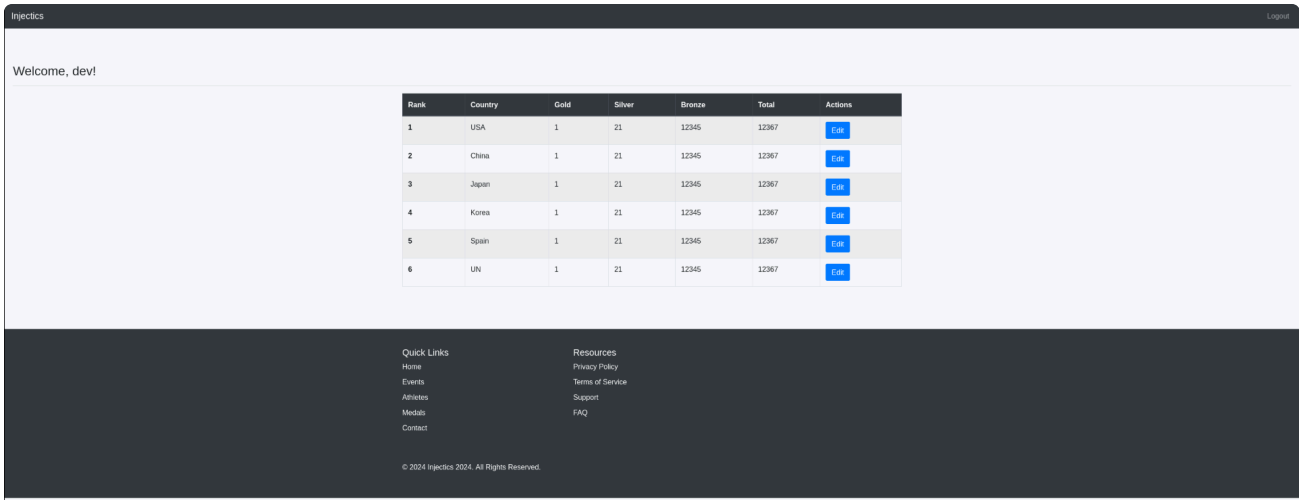
Impacto: Dropamos a tabela users para resetar a senha do usuário superadmin e logar com ele no admin panel, permitindo que possamos usufruir de suas funcionalidades.

Conselhos e Remediações: Qualquer entrada nesse formulário deve ser colocada na consulta sql como valor e não como parte da consulta SQL , exigindo que utilize

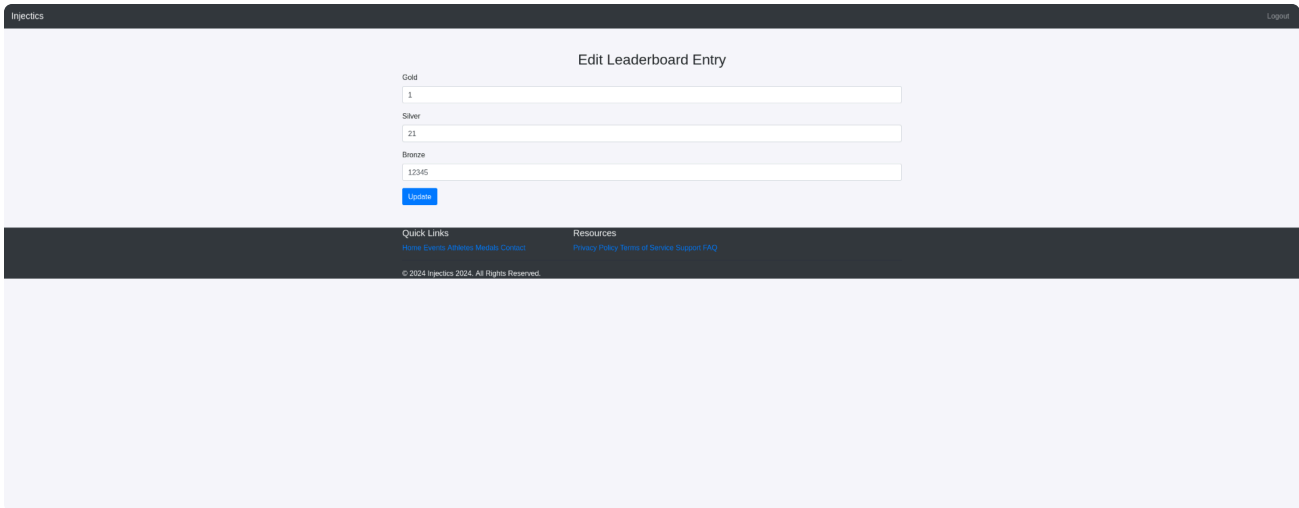
consultas sql statement com bind_param.

Detalhes Técnicos e Evidências:

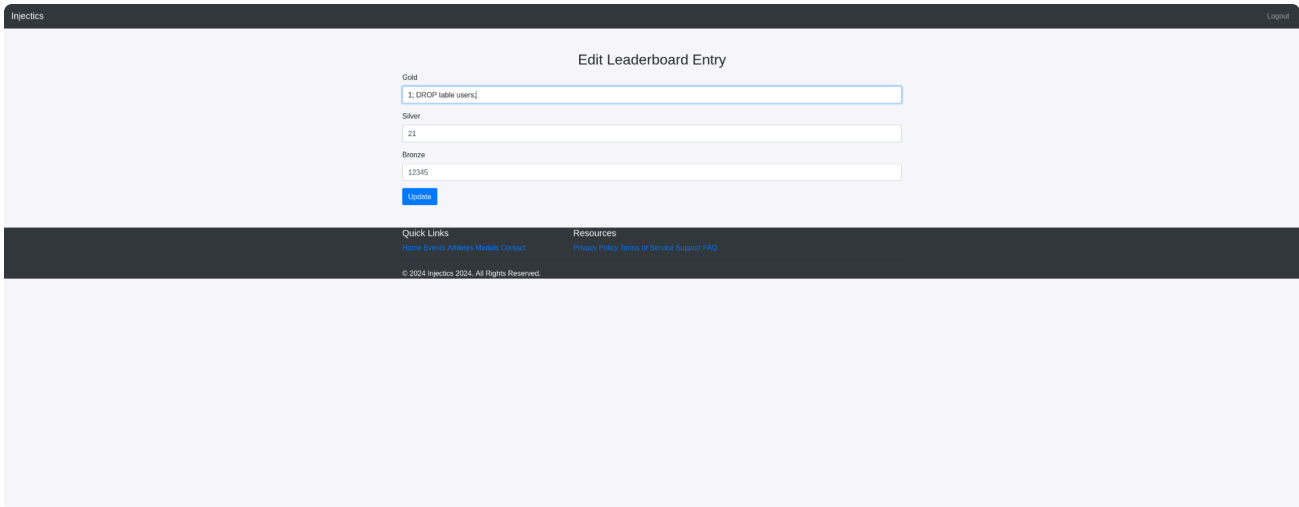
1. Estamos no dashboard como dev.



2. Apertamos para editar qualquer registro.



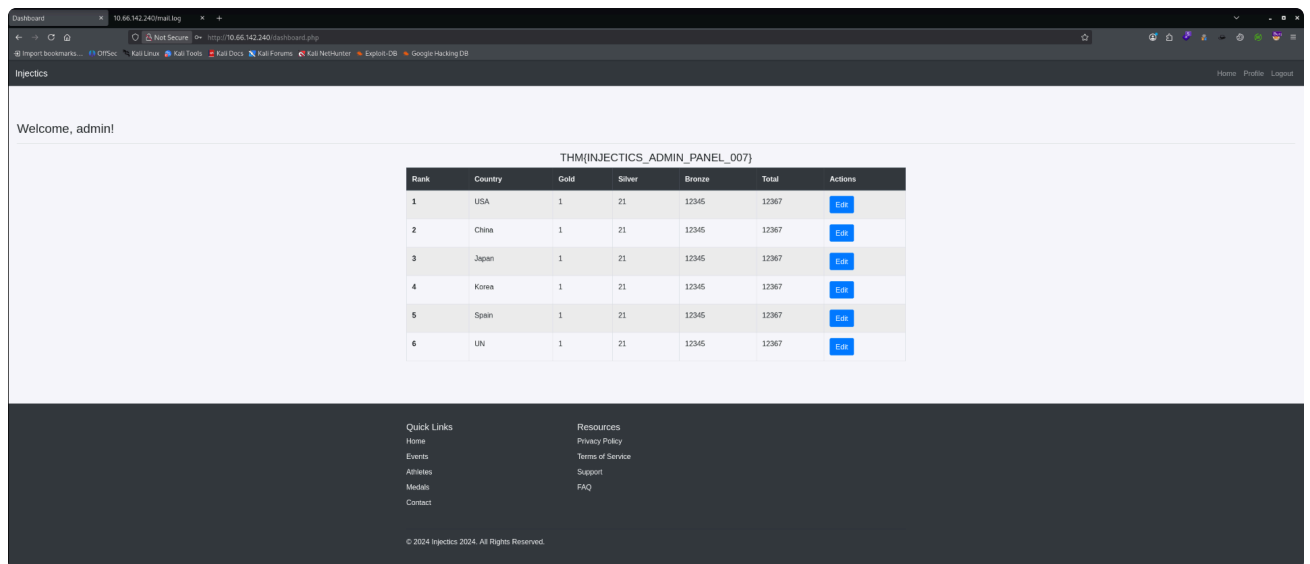
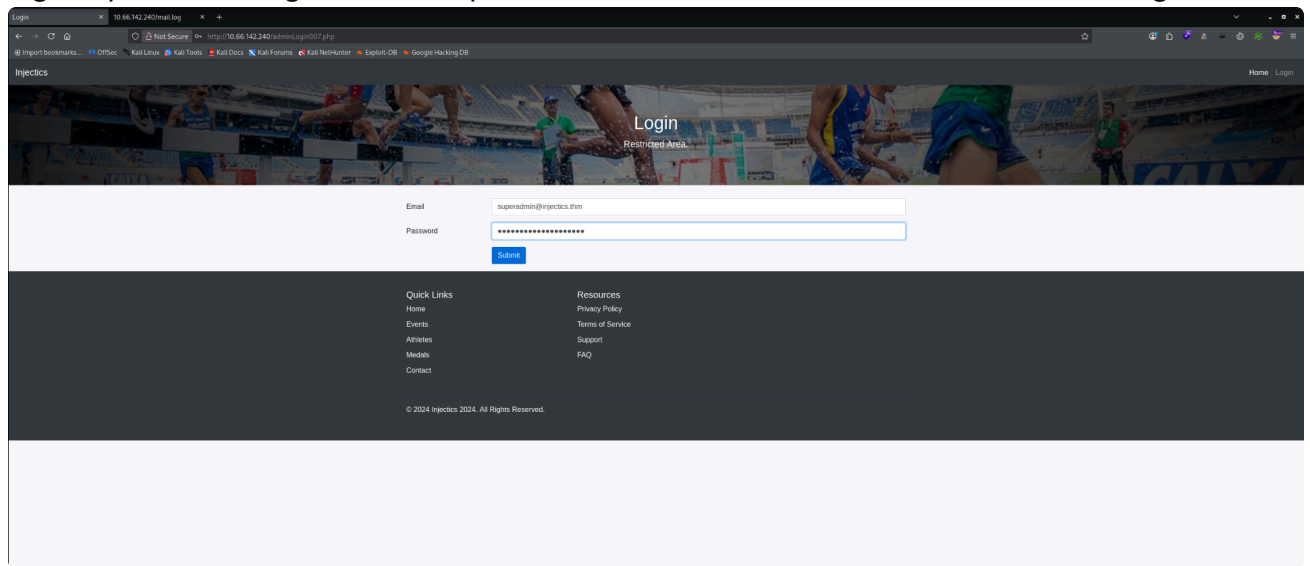
3. No campo gold colocaremos o seguinte payload.



4. Apertamos update e o servidor crash dizendo que irá restaurar o banco para recuperar a tabela excluída.



5. Agora podemos logar como superadmin com a senha encontrada em mail.log



✗ Vulnerabilidade encontrada

Título: SSTI

Risk Rating: 10 DREAD

Resumo: Foi possível fazer um SSTI em um parâmetro firstname no update profile do admin usando o twig, assim conseguimos injetar um reverse shell

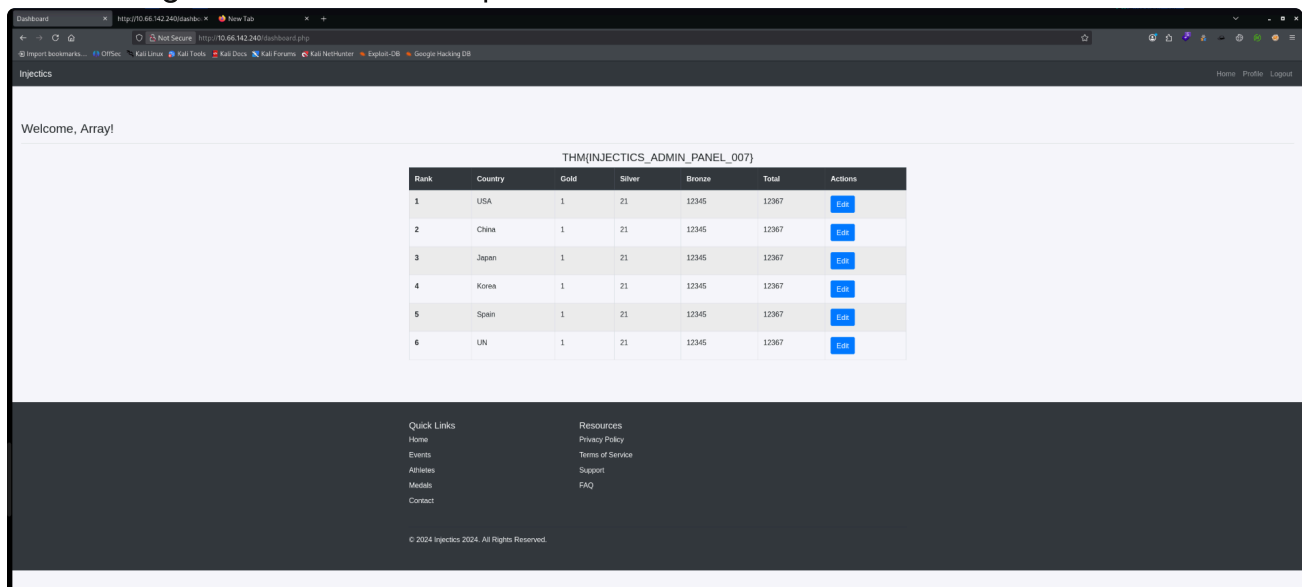
Background(Contexto adicional): Um SSTI é uma injeção feita em mecanismos de template para gerar conteúdo dinâmico em uma página html, porém os dados que normalmente chegam nele não são bem tratados permitindo injeção de entradas maliciosas.

Impacto: Com o SSTI foi possível gerar reverse shell e entrar no servidor web como www-data, possibilitando possíveis escalasões de privilégio, exfiltração de dados de usuários e do contexto da aplicação web.

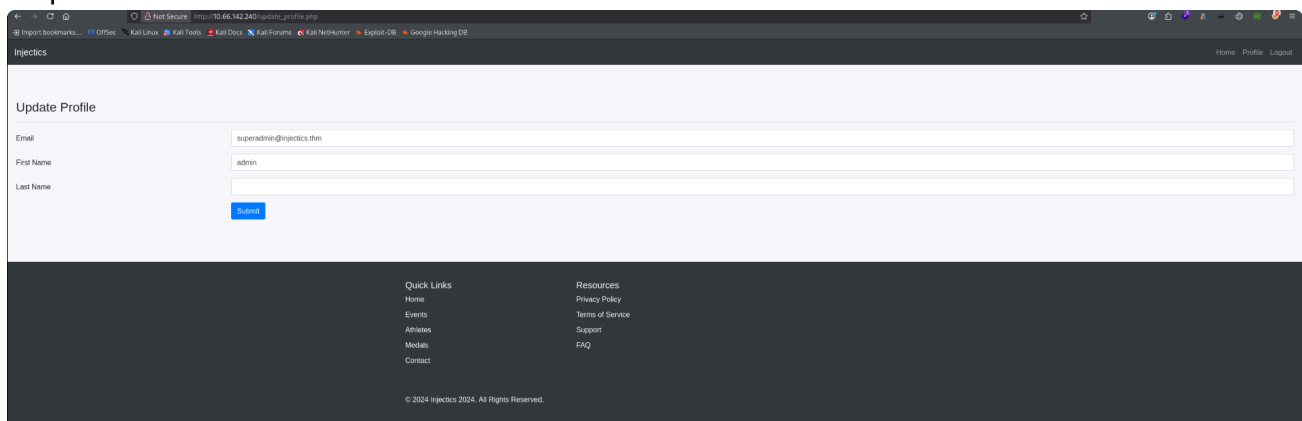
Conselhos e Remediações: Esse campo não deve permitir que a entrada seja processada como código, assim se for utilizado `{{}}` na entrada então deve ser tratado como texto e não comando.

Detalhes Técnicos e Evidências:

1. Quando logamos como admin aparecemos no dashboard



2. Depois temos de ir em Profile.



3. Injetamos esse payload da foto a seguir onde ip pode ser substituído pelo ip da máquina da qual você está ouvindo e a porta pode ser modificada.

4. Apertamos submit e depois vamos para home. A tela irá travar e o shell será criado.

```
$ penelope -p 4444
[*] Listening for reverse shells on 0.0.0.0:4444 -> 127.0.0.1 - 10.0.2.15 - 172.17.0.1 - 192.168.129.186
[*] Main Menu (m) Payloads (p) Clear (Ctrl-L) Quit (q/Ctrl-C)
[*] [New Reverse Shell] => injectics 10.66.142.240 Linux-x86_64 www-data(33) Session ID <1>
[*] ★ Agent deployed via /usr/bin/python3
[*] Interacting with session [1] - PTY - Menu key F12 =
[*] Session log: /home/kali/.penelope/sessions/injectics-10.66.142.240-Linux-x86_64/2026_09_28-22_27_30-467-www-data(33).log

www-data@injectics:/var/www/html$ ls
adminlogin007.php  composer.lock  css  dashboard.php  flags  index.php  js  logout.php  script.js  update_profile.php
banner.jpg         composer.lock  css  edit_leaderboard.php  functions.php  injecticsService.php  login.php  mail.log  styles.css  vendor
www-data@injectics:/var/www/html$ cd flags
www-data@injectics:/var/www/html/flags$ ls
5d8af1dc14503c7e4bdc8e51a3469f48.txt
www-data@injectics:/var/www/html/flags$ cat 5d8af1dc14503c7e4bdc8e51a3469f48.txt
THM{5735172bec147f4dd649872f73e0fdea}
```

Tabela de Credenciais Capturadas

Serviço / Local	Usuário	Senha / Hash	Origem do Achado
User web	dev@injectics.thm	devPasswd123	mail.log
User web	superadmin@injectics.thm	superSecurePasswd101	mail.log

Considerações Técnicas & Cheat Sheet

Lições Aprendidas

1. Testar sempre consultas codificadas e não codificadas , perdi muito tempo no sql injection para dropar users pois eu estava enviando as consultas de forma codificada e utilizando ' , gerando erros desnecessários.

2. Procurar sempre ver na aplicação se há campos dinâmicos que mudam de acordo com um valor mudado em um form por exemplo , o html pode estar usando mecanismos de template como twig e podem ser campos vulneráveis.